Report for the Social Media Mining (CSE598)

Huajian Liu
Arizona State University
Tempe, US
huajianl@asu.edu

Abstract—This report is intended to summarize the three projects from the Social Media Mining course.

I. Introduction

The projects in this course covered implementations from two perspectives. One involves algorithms related to social media mining, while the other focuses on services and applications based on SOC (service-oriented computing).

II. Project 1: Social Network Measurements

A. Project Description

The aim of this project is to ensure that we understand and are familiar with the concepts covered in the lectures, including graphs, graph representation, connection, degree, and centrality. By the end of the project, we should have applied these concepts in generating graphs and computing network measurements both manually and through programming.

There are two parts to this project. The first part involves drawing the directed graph and creating the adjacency matrix of the directed graph. All data will be sourced from a simple website. The second part involves implementing the algorithms for eigenvector centrality.

B. Drawing the directed graph

We manually create a social networking graph from a specified website. The graph is a (weakly or strongly) connected directed graph with 12 nodes and 17 edges (arcs). Start constructing the graph from the root website https://www.public.asu.edu/~ychen10/teaching/honors.html. Each website (page) will be represented as a node in the graph, and each URL linking one webpage to another will be an edge in the graph. Stop expanding the graph once the required number of nodes and edges is reached.

Image 1 the directed graph

C. Making the adjacent matrix of the graph

An adjacency matrix is a way of representing a graph using a square matrix. For a directed graph with N nodes, the adjacency matrix is an $n \times n$ matrix A, where $A[i][j]$ indicates whether there is a directed edge from node i to node j.

Here's how to construct an adjacency matrix from a directed graph:

- **Initialize the Matrix**:
  Create an $n \times n$ matrix A filled with zeros, where n is the number of nodes in the graph.
- **Fill the Matrix**:

For each directed edge from node i to node j, set $A[i][j]$ = 1.

If there is no edge from node i to node j, $A[i][j]$ remains 0.

Image 2 the adjacency matrix

```
0,1,1,1,1,1,1,0,0,0,0,0
1,0,0,0,0,0,0,0,0,0,0,0
1,0,0,0,0,0,0,1,0,0,0,0
1,0,0,0,0,0,0,0,0,0,0,0
1,0,0,0,0,1,0,0,0,0,0,0
0,0,0,0,1,0,0,0,0,0,1,0
0,0,0,0,0,0,0,0,1,1,1,1
0,0,0,0,0,0,0,0,0,0,0,1
0,0,0,0,0,0,0,0,0,0,0,1
0,0,0,0,0,0,0,0,0,0,0,1
0,0,0,0,0,0,0,0,0,0,0,1
0,0,0,0,0,0,0,0,0,0,0,0
```

D. The implement of betweenness_centrality with python

We use a function calculates the betweenness centrality of a graph represented by an adjacency matrix stored in a CSV file. Here is a description of the functionality and the steps implemented in the code:

1) Functionality:

1. **Read CSV File**: Reads a CSV file containing an adjacency matrix that represents a graph.
2. **Create Graph**: Constructs a graph `G` using NetworkX library from the adjacency matrix.
3. **Calculate Betweenness Centrality**: Computes the betweenness centrality of each node in the graph.
4. **Return Results**: Returns a dictionary where the keys are nodes and the values are their corresponding betweenness centrality scores.

Here is the python code

```
def betweenness_centrality(csv_file):
        # Extract the matrix from the CSV file
    adjacency_matrix = []
    with open(csv_file, 'r') as file:
        reader = csv.reader(file)
        for row in reader:
            row = [int(val) for val in row]
            adjacency_matrix.append(row)
    adjacency_matrix = np.array(adjacency_matrix)

    # create the graph from the matrix
    G = nx.from_numpy_array(adjacency_matrix)

    # calculate the betweenness centrality of the
graph
    betweenness_centrality =
nx.betweenness_centrality(G)

    return betweenness_centrality

print(betweenness_centrality("submission.csv"))
```

In this experiment we use the data below as the input:

```
0,1,1,1,1,1,1,0,0,0,0,0
1,0,0,0,0,0,0,0,0,0,0,0
1,0,0,0,0,0,0,1,0,0,0,0
1,0,0,0,0,0,0,0,0,0,0,0
1,0,0,0,0,1,0,0,0,0,0,0
1,0,0,0,1,0,0,0,0,0,1,0
1,0,0,0,0,0,0,0,1,1,1,1
0,0,1,0,0,0,0,0,0,0,0,1
0,0,0,0,0,0,1,0,0,0,0,1
0,0,0,0,0,0,1,0,0,0,0,1
0,0,0,0,0,1,1,0,0,0,0,1
0,0,0,0,0,0,1,1,1,1,1,0
```

and we obtain the results as shown below:

TABLE I. Experiment Results

Model Name	Index of item											
	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*
Betweenness centrality	***0.54***	***0***	***0.08***	***0***	***0***	***0.08***	***0.31***	***0.04***	***0***	***0***	***0.07***	***0.15***

The results are rounded to two decimal places using rounding rules (rounding half up).

E. What have we learned

Betweenness centrality measures a node's importance based on how often it acts as a bridge along the shortest path between two other nodes. High betweenness centrality indicates a node's critical role in facilitating communication and connectivity within the network. Nodes with high betweenness centrality can control information flow, making them influential in network dynamics and essential for maintaining the network's structural integrity.

III. Project 2: Implementation of SOC STRUCTURE

A. Project Description

The project consists of two main components: a server-side and a client-side. The server-side, implemented using Python Flask, is located in the Project2 directory, while the client-side, built with C# ASP.NET, resides in the WebApplicationQution2 directory. The Visual Studio startup file is found at Project2/Project2.sln. Additionally, three files related to Problem 2, named file1, file2, and file3, are stored in the current user's Downloads directory.

B. Implementation of the server-side

This Flask-based server application is initialized with the Flask framework and includes several routes and functions for web content manipulation. The /getWebContent route retrieves web content from a given URL using the web2string function. The establish_https_connection_with_retry function establishes an HTTPS connection with a retry mechanism. The /wordFilter route filters stop words and punctuation from the provided content using BeautifulSoup for HTML parsing and the wordFilter function for cleaning the text. A predefined set of stop words is used for filtering. The /top10Words route identifies the top 10 most common words from the filtered content using the top10words function and the Counter class from the collections module. The server runs on localhost at port 4999.

The main code as below

```
# -*- coding: utf-8 -*-
from flask import Flask, request, jsonify
import http.client
import requests
# Init server with Flask
app = Flask(__name__)

 Route to get web content
@app.route('/getWebContent', methods = ['GET'])
def GetWebContent():
    url = request.args.get('url')
    if not url:
        return jsonify({'error': 'URL parameter is missing'}), 400
    content = web2string(url)
     # Return the content of the web page
    return content
# Route to filter words from content
@app.route('/wordFilter', methods = ['GET'])
def WordFilter():
    content = request.args.get('content')
    soup = BeautifulSoup(content, "html.parser")
    # extract all text in the html web
    textContent = soup.get_text(separator=' ', strip=True)
    filtedContent = wordFilter(textContent)
    return filtedContent
# Route to get the top 10 words from content
@app.route('/top10Words', methods = ['GET'])
def Top10Words():
    content = request.args.get('content')
    mostCommon = top10words(content)
    return mostCommon

# Start server at port 4999
if __name__ == '__main__':
    app.run('localhost', 4999)
```

C. Implementation of the client-side

This application demonstrates how to utilize web scraping, text processing, and TF-IDF calculations in a web environment. It combines multiple functionalities to fetch web content, filter words, generate files, and compute TF-IDF scores, showcasing a practical application of text analytics in a web context.

D. What have we learned

The SOC (Service-Object-Structure) architecture is a structured approach that promotes modular, maintainable, and scalable software design. By separating concerns into Service, Object, and Structure layers, SOC improves clarity, flexibility, and adaptability in developing various types of software systems, from enterprise applications to distributed microservices architectures.

IV. Project 3: The algorithms implemented for social media data mining

A. Project Description

In this project,we implement 10 algorithms for social media data mining. They are maxClique, clique percolation method(CPM),node_similarity,modularity,community_density,girvan_newman,max_cascade,cosineSimilarity,content_based_recommendation,cf_recommendation_k1, and the sis_model.

B. Implementation of maxClique

The maxClique function finds the largest clique (fully connected subgraph) in an undirected graph represented by an adjacency matrix. It starts by determining the number of nodes in the matrix. It then searches for the largest clique by iterating from the maximum possible clique size down to 1. In each iteration, it generates all possible node combinations of the current size and checks if they form a clique using the isClique function. If a valid clique is found, it is saved as maxClique. Finally, the function returns the nodes in the largest clique as a space-separated string, with node indices starting from 1.

C. Implementation of CPM(clique percolation method)

The CPM function identifies all sub-communities in a graph using the Clique Percolation Method (CPM). Here's a simplified explanation of its functionality:The function begins by determining the number of nodes in the graph from the adjacency matrix matrix. It then searches for all cliques of a specified minimum size k by generating all possible combinations of k nodes and checking if each combination forms a clique using the isClique function. These cliques are stored in the allCliques list. Next, the function merges these cliques into larger sub-communities using the mergeCliques function. Finally, it formats these sub-communities as strings, where each sub-community is represented by node numbers sorted in ascending order and enclosed in curly braces. The function returns these formatted sub-communities as a single comma-separated string.

D. Implementation of node_similarity

The node_similarity function calculates the Jaccard similarity between the first two nodes of an undirected graph represented by an adjacency matrix. Here's a simplified explanation of its functionality: The function starts by identifying the neighbors of the first two nodes using np.where to find the indices of non-zero elements in the respective rows of the matrix. These neighbors are stored in sets neighbors1 and neighbors2. The Jaccard similarity is then computed by taking the size of the intersection of these two sets (common neighbors) and dividing it by the size of their union (total unique neighbors), excluding the nodes themselves. The resulting similarity score is returned as a float.

E. Implementation of modularity

The modularity function calculates the modularity matrix for an undirected graph based on its adjacency matrix. It begins by computing the degree of each node, which is the sum of its connections. Then, it calculates the total number of edges in the graph by summing all node degrees and dividing by two. Using these values, the function computes the modularity matrix by subtracting the expected number of edges (determined by the outer product of node degrees divided by twice the total number of edges) from the actual adjacency matrix. The resulting modularity matrix, which highlights the difference between observed and expected connections, is returned as a NumPy array (np.ndarray).

F. Implementation of girvan_newman

The girvan_newman function calculates the edge betweenness centrality matrix for an undirected graph given its adjacency matrix. It first creates a NetworkX graph G from the adjacency matrix and calculates the edge betweenness centrality for each edge using NetworkX's edge_betweenness_centrality function. This centrality measure reflects the importance of edges based on the number of shortest paths that pass through them. The results are stored in a dictionary. The function then initializes a NumPy array edge_betweenness_matrix with dimensions equal to the number of nodes and populates this matrix with the calculated betweenness values, ensuring symmetry for the undirected graph. The resulting matrix, representing the edge betweenness centrality for all edges, is returned as a NumPy array.

G. Implementation of max_cascade

The max_cascade function identifies the node in a graph that can influence the maximum number of other nodes through a cascade effect, using a given activation threshold. It starts by converting the adjacency matrix into a NetworkX graph G and initializes variables to track the maximum number of influenced nodes and the best starting node. For each node, it performs a breadth-first search (BFS) to explore its neighbors based on the activation threshold in the matrix, adding neighbors to the influenced set if their activation threshold exceeds 0.5 and they haven't been influenced yet. This process continues until all reachable nodes are explored. The function updates the maximum influenced nodes and the best starting node whenever a node influences more nodes than previously recorded. Finally, it returns the node that can initiate the cascade affecting the most nodes.

H. Implementation of cf_recommendation

The cf_rec*ommen*dation_k1 function predicts unknown ratings in a user-item matrix using collaborative filtering. It starts by calculating cosine similarity between items and the average ratings for each item. With k set to 1, the function iterates through the matrix to identify unknown ratings

(marked as 9). For each unknown rating, it retrieves and sorts the item similarities, selecting the top-k similar items. The predicted rating is computed using a weighted sum of the known ratings of these similar items, adjusted by item averages and weighted by similarity scores. If the weighted sum's denominator is zero, the item average is used instead. The function stores and returns the predicted rating for the specific user-item pair.

REFERENCES

[1] Reza Zafarani, Mohammad Ali Abbasi, and Huan Liu,“ISocial Media Mining: An Introduction”

[2] Nikola Gemes,“Social Media Data Mining: What It Is, How It Works, and How to Use It”, online at: https://whatagraph.com/blog/articles/social-media-data-mining

[3] Alexis Baudin,“Clique percolation method: memory efficient almost exact communities”,online at: https://kirgizov.link/publications/BDKMG.pdf

[4] wikipedia,“Collaborative filtering”,online at: https://en.wikipedia.org/wiki/Collaborative_filtering#:~:text=Collaborative%20filtering%20(CF)%20is%20a,videos%2C%20images%2C%20games).